Rapport de notes : Gérer du code avec Git et GitHub

Formation OpenClassrooms « Gérez du code avec Git et GitHub » (Développement), progression 100 %, 3 objectifs pédagogiques sur 3.

Auteur : René DJONTU. Octobre 2026.

1. Introduction et objectifs

Ce rapport synthétise les notions étudiées au cours de la formation. Git est un système de gestion de versions décentralisé : il enregistre l'historique d'un projet, permet de revenir à un état antérieur, de travailler en parallèle sur plusieurs fonctionnalités et de collaborer sans écraser le travail des autres. GitHub est une plateforme d'hébergement de dépôts Git qui ajoute des outils de collaboration (pull requests, issues, revues de code).

Les compétences visées se regroupent en trois axes :

1. Suivre et sauvegarder l'évolution d'un projet avec Git en local.
2. Organiser le travail avec les branches et les fusions.
3. Collaborer à distance avec GitHub.

2. Concepts fondamentaux

Gestion de versions. Un système de gestion de versions conserve chaque état significatif du code, avec son auteur, sa date et son motif. Il remplace les copies manuelles de dossiers (projet_final_v2_corrige) par un historique fiable et consultable.

Architecture décentralisée. Chaque développeur possède une copie complète du dépôt, historique compris. Le travail local reste possible hors connexion, et la synchronisation avec un dépôt distant est une étape distincte.

Les trois zones de Git.

Zone	Rôle
Répertoire de travail (working directory)	Les fichiers tels qu'ils sont sur le disque, en cours de modification

Zone	Rôle
Zone d'index (staging area)	Les modifications sélectionnées pour le prochain commit
Dépôt local (repository)	L'historique des commits enregistrés

Vocabulaire essentiel.

- Commit : instantané du projet, identifié par un hachage unique, accompagné d'un message.
- Branche : pointeur mobile vers un commit, représentant une ligne de développement.
- HEAD : référence vers le commit ou la branche actuellement extrait.
- Dépôt distant (remote) : copie du dépôt hébergée sur un serveur, nommée par convention origin.

3. Configuration initiale

Avant le premier commit, il faut déclarer son identité, qui sera associée à chaque commit.

```
git config --global user.name "Prénom Nom"
git config --global user.email "adresse@exemple.com"
git config --global init.defaultBranch main
git config --list
```

L'authentification à GitHub se fait par jeton d'accès personnel en HTTPS ou par clé SSH. La clé SSH évite de ressaisir ses identifiants à chaque échange.

4. Cycle de travail local

Le cycle de base se répète tout au long du projet : modifier, sélectionner, enregistrer.

```
git init                    # créer un dépôt dans le dossier courant
git status                  # état des fichiers (modifiés, indexés, non
suivis)
git add fichier.py          # ajouter un fichier à l'index
git add .                   # ajouter toutes les modifications
git commit -m "Message"     # enregistrer l'index dans l'historique
```

Points à retenir :

- `git status` est la commande à utiliser le plus souvent : elle indique toujours la prochaine action possible.
- Un fichier doit être indexé avec `git add` pour entrer dans le prochain commit.
- Un commit est local : il n'est visible par les autres qu'après un `git push`.

5. Consulter l'historique

```
git log                       # historique complet
git log --oneline --graph     # vue condensée avec représentation des
branches
git diff                      # différences non indexées
git diff --staged             # différences indexées, prêtes à être commitées
git show <hash>               # détail d'un commit
```

La lecture régulière de l'historique permet de vérifier ce qui va être enregistré et de retrouver l'origine d'une modification.

6. Annuler et corriger

Situation	Commande
Retirer un fichier de l'index	`git restore --staged fichier`
Abandonner les modifications d'un fichier	`git restore fichier`
Corriger le dernier commit (message ou contenu)	`git commit --amend`
Annuler un commit en créant un commit inverse	`git revert <hash>`
Revenir à un commit antérieur en réécrivant l'historique	`git reset --hard <hash>`

Règle de prudence. `git revert` est sûr pour un historique déjà partagé, car il ajoute un commit sans rien effacer. `git reset --hard` supprime des modifications de façon définitive et ne doit pas être utilisé sur des commits déjà poussés vers un dépôt partagé.

7. Branches et fusions

Les branches permettent de développer une fonctionnalité ou de corriger un bogue sans toucher à la branche principale. Leur création est quasi instantanée, car une branche n'est qu'un pointeur.

```
git branch                    # lister les branches
git switch -c ma-fonctionnalite   # créer et basculer sur une branche
git switch main               # revenir sur main
git merge ma-fonctionnalite   # fusionner dans la branche courante
git branch -d ma-fonctionnalite   # supprimer une branche fusionnée
```

Types de fusion.

- Fusion en avance rapide (fast-forward) : la branche principale n'a pas évolué, Git déplace simplement le pointeur.
- Fusion avec commit de fusion : les deux branches ont divergé, Git crée un commit à deux parents.

Gestion des conflits. Un conflit apparaît lorsque deux branches modifient les mêmes lignes. Git marque les zones concernées dans le fichier (`<<<<<<<` , `=======` , `>>>>>>>`). La résolution consiste à :

1. Ouvrir le fichier et choisir le contenu final.
2. Supprimer les marqueurs.
3. Indexer le fichier avec `git add` .
4. Terminer avec `git commit` .

Rebase. `git rebase main` rejoue les commits d'une branche au-dessus de la dernière version de main pour obtenir un historique linéaire. Il réécrit l'historique : à éviter sur des branches déjà partagées.

8. GitHub et dépôts distants

```
git clone <url>                          # copier un dépôt distant en local
git remote add origin <url>              # relier un dépôt local à un dépôt
distant
git remote -v                            # lister les dépôts distants
git push -u origin main                  # envoyer les commits (première fois,
avec suivi)
git push                                 # envois suivants
git fetch                                # récupérer les nouveautés sans les
fusionner
git pull                                 # fetch suivi d'une fusion
```

Distinction utile : `git fetch` met à jour la connaissance du dépôt distant sans modifier le travail en cours, tandis que `git pull` intègre immédiatement les changements dans la branche courante.

9. Collaboration avec GitHub

Flux de travail par branches et pull requests.

1. Créer une branche dédiée à la tâche.
2. Commiter régulièrement, puis pousser la branche vers GitHub.
3. Ouvrir une pull request pour proposer l'intégration dans main.
4. Faire relire le code : commentaires, demandes de modification, approbation.
5. Fusionner la pull request puis supprimer la branche.

Contribuer à un projet tiers. On crée un fork (copie du dépôt dans son compte), on clone ce fork, on travaille sur une branche, puis on ouvre une pull request vers le dépôt d'origine.

Autres outils GitHub.

- Issues : suivi des bogues, tâches et idées, avec étiquettes et assignations.
- Fichier README : présentation du projet, installation, utilisation.
- Licence : cadre d'utilisation et de réutilisation du code.
- Profil et dépôts publics : vitrine professionnelle pour un développeur.

10. Bonnes pratiques

- Faire des commits atomiques : une modification logique par commit.

- Rédiger des messages clairs, à l'impératif, avec un résumé court (environ 50 caractères) puis, si besoin, un corps explicatif.
- Ne jamais commiter de secrets (mots de passe, clés d'API, fichiers .env).
- Utiliser un fichier `.gitignore` pour exclure les fichiers générés, les dépendances et les données locales.
- Garder la branche main stable et travailler sur des branches nommées de façon explicite (par exemple `feature/connexion`, `fix/calcul-total`).
- Récupérer les changements distants (`git pull`) avant de pousser, pour limiter les conflits.
- Relire `git status` et `git diff --staged` avant chaque commit.

11. Aide-mémoire des commandes

Objectif	Commande
Initialiser un dépôt	`git init`
Cloner un dépôt	`git clone <url>`
Voir l'état	`git status`
Indexer	`git add <fichier>`
Enregistrer	`git commit -m "message"`
Historique	`git log --oneline --graph`
Créer une branche	`git switch -c <nom>`
Fusionner	`git merge <branche>`
Envoyer	`git push`
Récupérer	`git pull`
Annuler un commit partagé	`git revert <hash>`

12. Conclusion

La formation a permis de maîtriser le cycle complet d'un projet versionné : initialisation, enregistrement des modifications, organisation par branches, résolution des conflits, puis partage et collaboration via GitHub. Ces compétences s'appliquent directement aux projets de développement en équipe, où la traçabilité des changements, la revue de code et l'intégration progressive du travail sont indispensables.